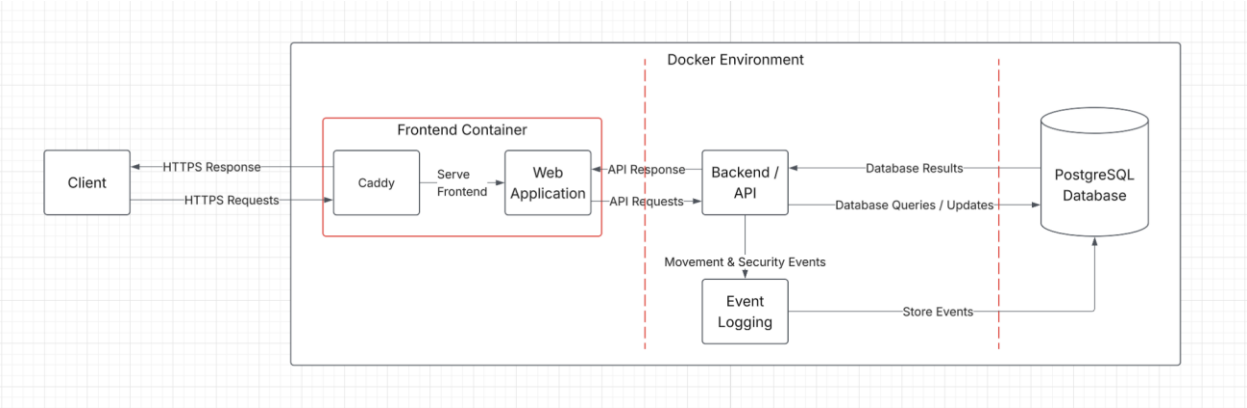
Design Document – Group 7

James Markijohn and Samaa Hediya

Software Architecture

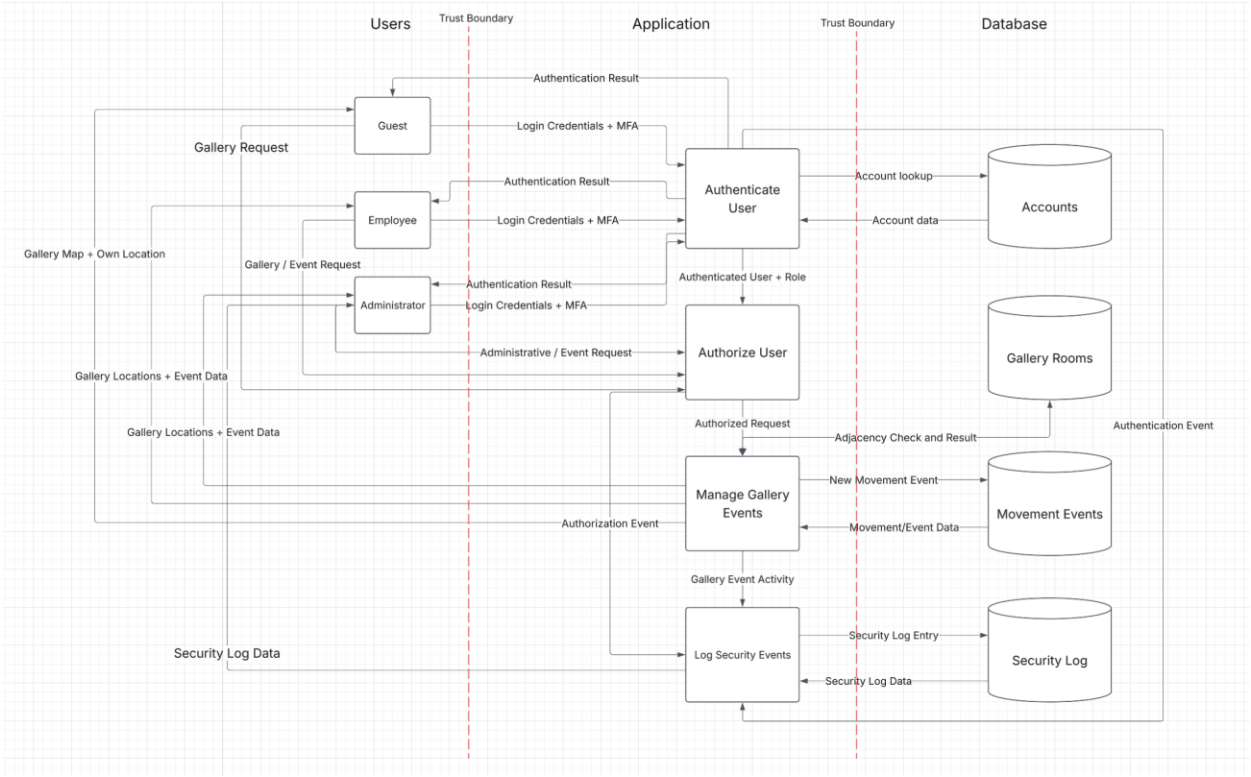


The application uses both client-server and event-driven architecture. The client connects to the application using HTTPS. Caddy works as the reverse proxy and serves the ReactJS web application. The web application sends API requests to the backend, which is built using Python and Django REST Framework. The backend then communicates with the PostgreSQL database to query or update data. The database sends the requested results back through the backend.

The event-driven part is used for movement and security events. When an event happens, such as a user moving between rooms or a security-related action, the backend sends the event to the event logging system, and the event is stored in the PostgreSQL database.

The application runs in a Docker environment. The database is not directly accessible from the internet, and users must communicate with it through the application and backend. The trust boundaries in the diagram show where data moves between different parts of the system. HTTPS is also used to protect data when it is sent between the client and the application.

Data Flow Diagram DFD



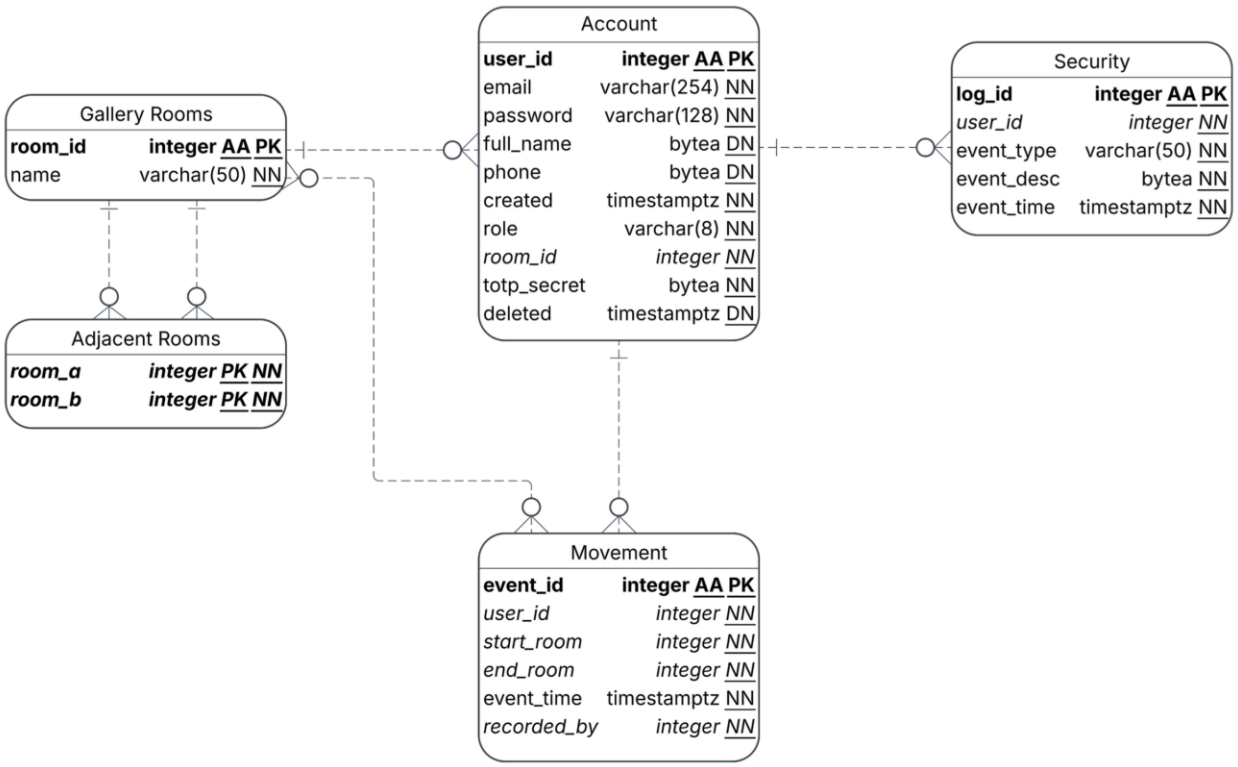
The Data Flow Diagram shows how information moves between the users, application, and database. There are three types of users: Guest, Employee, and Administrator. Users first send their login credentials and MFA information to the authentication process. The application checks the account information and then sends the authenticated user and their role to the authorization process. The authorization process decides what the user is allowed to access based on their role.

After the user is authorized, they can make requests to the gallery system. The application manages gallery events and communicates with the Gallery Rooms and Movement Events data. Guests have limited access and can only see the gallery map and their own location, while employees and administrators have more access to gallery locations and event information. These access levels follow the role-based permissions defined in the system requirements.

The system also records security-related activity. Authentication, authorization, and gallery activities can create security events, which are stored in the Security Log. The security log can only be accessed by administrators and cannot be modified by users. Pasted text

The trust boundaries separate the users, application, and database. They show where data moves from one part of the system to another and where security checks are important.

Entity Relationship Diagram (ERD)



This diagram describes the relationships between the database tables and the entities that they represent. Every user account will be stored in the accounts database. The user_id will be the primary key (auto-incremented integer), and the required fields (not null) are the email address, password, created, totp_secret, and role fields. The password field will be a hash, the role will automatically be “guest”, and the room_id field (a foreign key) will show which room the user is in. Upon deletion, values will be pseudonymized and a deleted timestamp will be added. If the deleted value is not null, this indicates to the web app that the account has been deleted and should not be able to be used.

The security log table will contain a log_id as the primary key, a user_id as a foreign key, and an event_type (failed login, failed MFA, database query, etc.), event_desc (description), and event_time. There is a one-to-many relationship between the accounts and security log tables: one account may generate many security logs, but one log may only be affiliated with one user. The description of the event will be encrypted to keep the specific details of any given event confidential.

The movement table holds movement events, with event_id as the primary key, user_id as a foreign key to the accounts database, event_time, and gallery rooms table foreign keys start_room and end_room to show where the user moved to and from. There is a one-to-many

relationship between the accounts and movements tables: one account may move many times, but one movement may not be associated with many users. Additionally, there is a many-to-many relationship between this table and the gallery rooms table, as one event can consist of many rooms, and any room may be associated with many movement events.

The gallery rooms table will contain basic information about each room in the gallery, including a room_id (primary key), and the human-readable name of the room. There is a many-to-one relationship between the accounts table and this table, since many accounts may be in one room, but one account may not be in many rooms at once.

The adjacent rooms table will contain a composite primary key, which will consist of two foreign keys that reference the gallery rooms table. room_a and room_b indicate that if there is a record where two room IDs are the values for these fields, those two rooms are adjacent. The relationship between this table and the gallery rooms table is many-to-one, since there may be many adjacencies, but only one room may be used at a time for either of the foreign keys.

Threat Modeling with STRIDE

Spoofing	• Impersonation of real individuals (such as administrators or employees) during guest account creation. o Cannot verify the identity of users who create guest accounts without ID verification methods, but guest accounts will have only the minimum privileges applied, so impersonation will not be an effective threat vector. • Attackers may attempt to insert forged records into the movement event and security log database tables, including the identity of the account that is associated with those events. o Databases can only be modified through queries sent through a backend API, which will not be made accessible to guest users; attackers would need the administrator role to perform this attack. Additionally, administrators can edit databases to remove any forged records if they are detected.		
Tampering	Impersonation of real individuals (such as administrators or	Account creation system	Cannot verify the identity of users who create guest

	employees) during guest account creation.		accounts without ID verification methods, but guest accounts will have only the minimum privileges applied, so impersonation will not be an effective threat vector.
Spoofing	Attackers may attempt to insert forged records into the movement event and security log database tables, including the identity of the account that is associated with those events.	PostgreSQL database	Databases can only be modified through queries sent through a backend API, which will not be made accessible to guest users; attackers would need the administrator role to perform this attack. Additionally, administrators can edit databases to remove any forged records if they are detected.
Repudiation	A user may attempt to modify data sent to the application, such as changing movement information to record an invalid location or movement.	PostgreSQL database and application backend	Databases may only be accessed via a backend API, which will not be accessible to guest users. Parameterized queries will also mitigate the chances of unauthorized modifications of the movement event database. A compromised employee or administrator account can bypass these protections.
Information Disclosure	A user may perform an unauthorized action within the application, or	Any component of	The security log will contain any events

	attempt to perform a dangerous operation, then claim that they did not do so. • The location data for each guest and employee in the gallery will be visible on the map in the web interface of the employee or administrator accounts. Attackers may gain access to these maps if they compromise a privileged accounts. o MFA helps mitigate the instances of compromised passwords, though if an attacker gained access to a privileged account, there would be no mechanism to prevent them from viewing the map with the location data included.	the web application, backend and database in particular	that were triggered by specific user actions, which will allow administrators to provide non-repudiation. This also applies to the application's movement event database, which will contain all the movements of users; if a guest enters a zone that is out-of-bounds to them, this database will show that.
Denial of Service	Unauthorized database access could result in the entire contents of the application's databases being read and/or exfiltrated.	PostgreSQL database	This can be mitigated through preventing SQL injection through the use of parameterized queries, enforcing authorization based on role-based access for user accounts, and enforcing MFA to prevent instances of compromised accounts with higher levels of privilege. A trust boundary exists between the application and the databases, facilitated through the backend API.

Information Disclosure	The location data for each guest and employee in the gallery will be visible on the map in the web interface of the employee or administrator accounts. Attackers may gain access to these maps if they compromise a privileged accounts.	Frontend user interface (specifically administrator and employee views)	MFA helps mitigate the instances of compromised passwords, though if an attacker gained access to a privileged account, there would be no mechanism to prevent them from viewing the map with the location data included.
Elevation of Privilege	An attacker may send a large number of requests to the application in a short period of time, causing the server to become overloaded and preventing legitimate users from accessing the application. o An attacker may compromise an account to gain elevated privileges, though MFA is implemented to mitigate this type of attack.	Authentication system	Clients that send repeated failed login requests to this web application will be prevented from sending any additional requests for a short period of time (which balances slowing attackers without unduly inconveniencing users who forgot their credentials). The backend API that is uses to interface with the database will be rate limited to, for example, ten requests per account per minute. Testing will be performed during development to determine an appropriate rate limit.
Denial of Service	Attackers may attempt to provide invalid input to fields such as the account creation or login pages. This input could include SQL queries and	Any fields on frontend pages where user input is	Parameterized queries are being utilized to mitigate SQL injections, and

	large strings of characters, both of which could cause the system to lose data or crash, denying legitimate users from using the service.	accepted, especially account creation and login pages	input validation will be performed whenever input is accepted from users. For example, email addresses must be a certain number of characters in length (with lower and upper bounds) and be constructed according to specific syntax.
STRIDE Category	A user may attempt to gain permissions that belong to a higher-level role (such as a guest gaining employee privileges, or an employee gaining administrator privileges).	Authorization mechanisms in the backend	The role of the user will be inserted in the accounts table of the database and cannot be changed by anyone without administrator-level privileges. Guest user accounts should not be able to change their roles (or directly access the database at all, for that matter). An attacker may compromise an account to gain elevated privileges, though MFA is implemented to mitigate this type of attack.